

MongoDB

Irene Siragusa, PhD

Outline

-  MongoDB Characteristics
-  MongoDB Commands
 -  Comparison operators
 -  Element-wise and logical operators
-  Relations between elements (1-1)
-  Relations between elements (1-N)
-  Model Tree structure
-  Aggregation

Mongo DB characteristics

- Document-oriented NoSQL Open Source database
- Flexible structure
- Highly scalability
- Data in JSON-like format BSON
 - JSON JavaScript Object Notation
 - BSON: Binary JavaScript Object Notation

Mongo DB characteristics

- Availability
 - MongoDB Atlas
Fully managed service for MongoDB deployments in the cloud
 - MongoDB Enterprise
Subscription-based, self-managed version of MongoDB
 - MongoDB Community
Source-available, free-to-use, and self-managed version of MongoDB
- Interaction
 - MongoDB shell (mongosh)
 - Driver for programming languages
pymongo for Python

Mongo DB characteristics

- Documents are stored in BSON format and are made up key-value pairs
- Documents *do not* share the same internal structure
- Documents can have different fields (columns)
- Each document is stored in a collection
- Collections share indexes' structures and made a database

Mongo DB characteristics

- The attribute `_id` is always present in a document
 - it is the primary key
 - Cannot be changed
 - Can be every data type, but no array
- Can be created automatically within document insert, in that case it is an `objectId` type, related to the document's creation time

```
{
  "_id" : "37010"
  "city" : "ADAMS"
  "pop" : 2660
  "state" : "TN"
  "councilman" : {
    "name": "John Smith"
    "address": "13 Scenic Way"
  }
}
```

Mongo DB commands

- CRUD operations
 - **C**reate, **R**ead, **U**ppdate, and **D**eleete

Object db as
interface to
the database

method name

```
db.Users.insert({name: "irene", country: "Italy"})
```

Collection name

Document, here explicitly,
can be also a command

```
{ "_id" : ObjectId("5063114bd386d8fadbd6b004"), "name" : "irene",  
  "country": "Italy" }
```

Mongo DB commands

- Create implicitly a collection within the first insert() operation

- Primary key is automatic added

```
db.users.insert_one( {  
    user_id: "abc123",  
    age: 55,  
    status: "A"  
})
```

- or explicitly

```
db.createCollection("users")
```


Mongo DB commands

SQL	MongoDB
<pre>CREATE TABLE users (id MEDIUMINT NOT NULL AUTO_INCREMENT, user_id Varchar(30), age Number, status char(1), PRIMARY KEY (id))</pre>	<pre>db.users.insert({ user_id: "abc123", age: 55, status: "A" })</pre>

Mongo DB commands

- Multiple documents can be added at the same time with insertMany() operation

```
db.users.insertMany([
    {
        user_id: "abc123",
        age: 55,
        status: "A"
    }, {
        user_id: »edf456",
        age: 30,
        status: "B"
    })
```

Mongo DB commands

- Retrieve all the elements from a collection, via find() method

`db.users.find(<criteria>, <projection>)`

- eg
 - `db.products.find( { qty: { $gt: 25 } } )`

Mongo DB commands

SQL	MongoDB
SELECT id, user_id, status FROM users	db.users.find({ }, { user_id: 1, status: 1 })
SELECT user_id, status FROM users	db.users.find({ }, { user_id: 1, status: 1, _id: 0 })
SELECT * FROM users WHERE status = "A"	db.users.find({ status: "A" })
SELECT user_id, status FROM users WHERE status = "A"	db.users.find({ status: "A" }, { user_id: 1, status: 1, _id: 0 })

Mongo DB commands

SQL	MongoDB
<pre>SELECT * FROM users WHERE status = "A" AND age = 50</pre>	<pre>db.users.find({ status: "A", age: 50 })</pre>
<pre>SELECT * FROM users WHERE status = "A" OR age = 50</pre>	<pre>db.users.find({ \$or: [{ status: "A" }, { age: 50 }] })</pre>

Mongo DB commands

SQL	MongoDB
<pre>SELECT * FROM users WHERE status = "A" ORDER BY user_id ASC</pre>	<pre>db.users.find({ status: "A" }) .sort({ user_id: 1 })</pre>
<pre>SELECT * FROM users WHERE status = "A" ORDER BY user_id DESC</pre>	<pre>db.users.find({ status: "A" }) .sort({ user_id: - 1 })</pre>

Mongo DB commands

SQL	MongoDB
SELECT COUNT(*) FROM users	db.users.find().count()
SELECT COUNT(user_id) FROM users	db.users.find({ user_id: { \$exists: true } }) .count()
SELECT COUNT(*) FROM users WHERE age > 30	db.users.find({ age: { \$gt: 30 } }) .count()

Comparison operators

Operator	Description
\$eq	Matches values that are equal to a specified value.
\$gt	Matches values that are greater than a specified value.
\$gte	Matches values that are greater than or equal to a specified value.
\$in	Matches any of the values specified in an array.
\$lt	Matches values that are less than a specified value.
\$lte	Matches values that are less than or equal to a specified value.
\$ne	Matches all values that are not equal to a specified value.
\$nin	Matches none of the values specified in an array.

Element-wise and logical operators

Operator	Description
\$and	Joins query clauses with a logical AND returns all documents that match the conditions of both clauses.
\$not	Inverts the effect of a query expression and returns documents that do not match the query expression.
\$nor	Joins query clauses with a logical NOR returns all documents that fail to match both clauses.
\$or	Joins query clauses with a logical OR returns all documents that match the conditions of either clause.
\$exists	Matches documents that have the specified field.
\$type	Selects documents if a field is of the specified type.

Mongo DB commands

- A field in a document can be added directly without schema modification, via update() method and \$set operator

```
db.users.update_many(  
    { },  
    { $set: { join_date: get_join_date() } },  
)
```

Mongo DB commands

- A field in a document can be removed via \$unset operator

```
db.users.update(  
    {},  
    { $unset: { join_date: " " } },  
)
```

- Collection can be deleted via drop method

```
db.users.drop()
```

Mongo DB commands

- Update certain elements in a collection with update() method

`db.collection.update(<query>, <update>, <upsert>, <multi>)`

- upsert
 - If true creates a new document if no documents match the query, or updates a single document that matches the query.
- multi
 - If true, updates all documents that meet the query criteria. If false, limit the update to one document that meet the query criteria.

Mongo DB commands

SQL	MongoDB
UPDATE users SET status = "C" WHERE age > 25	<pre>db.users.update({ age: { \$gt: 25 } }, { \$set: { status: "C" } }, { multi: true })</pre>
UPDATE users SET age = age + 3 WHERE status = "A"	<pre>db.users.update({ status: "A" }, { \$inc: { age: 3 } }, { multi: true })</pre>

Mongo DB commands

- Delete certain elements in a collection with delete() method

```
db.collection.delete(<query>)
```

SQL	MongoDB
DELETE FROM users WHERE status = "D"	db.users.delete({ status: "D" })

Relations between elements (1-1)

```
Person {  
  _id: "joe",  
  name: "Joe Bookreader"  
}
```

```
Address {  
  person_id: "joe",  
  street: "123 Fake Street", city: "Faketon",  
  state: "MA",  
  zip: "12345"  
}
```

```
Person {  
  _id: "joe",  
  name: "Joe Bookreader"  
  address: {  
    street: "123 Fake Street",  
    city: "Faketon",  
    state: "MA",  
    zip: "12345"  
  }  
}
```

Relations between elements (1-N)

```
Person { _id: "joe",  
        name: "Joe Bookreader"  
}
```

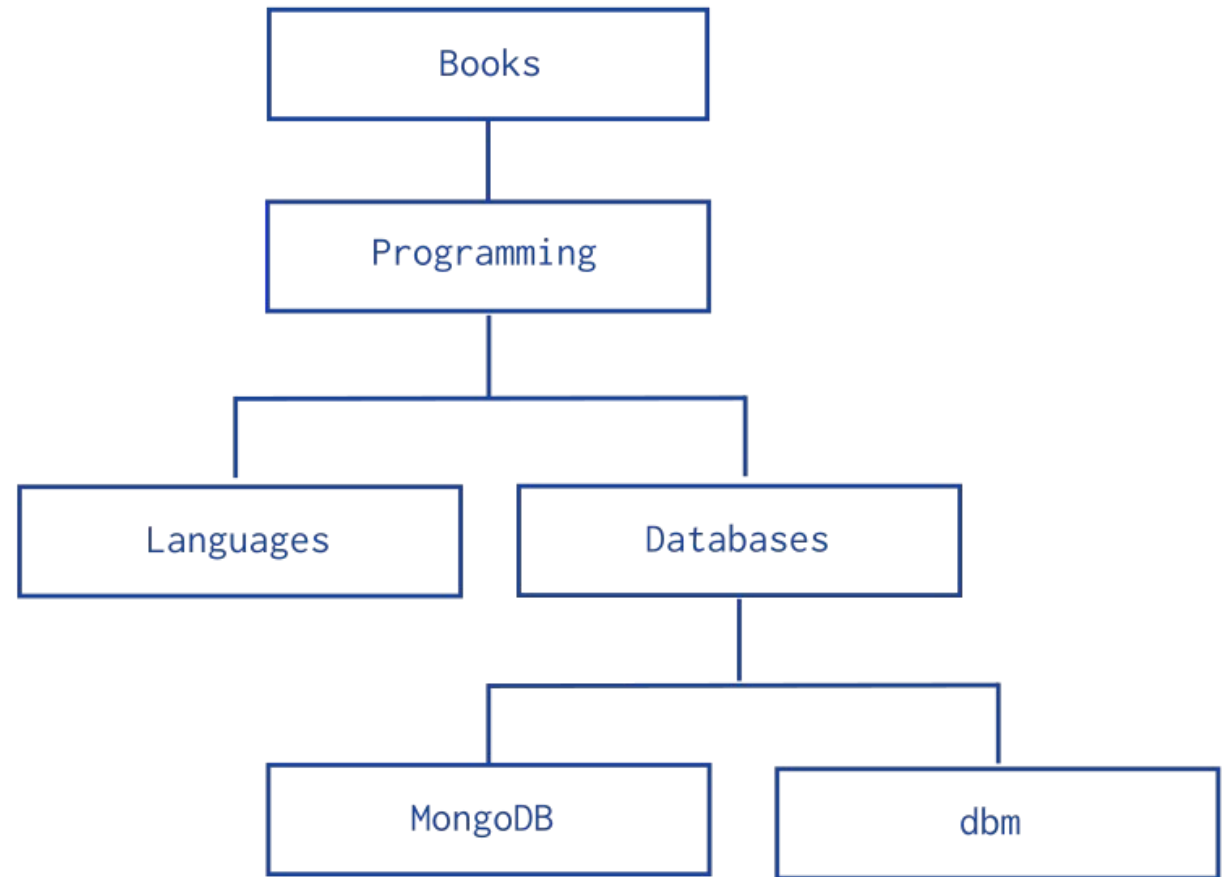
```
Address { person_id: "joe",  
          street: "123 Fake Street",  
          city: "Faketon",  
          state: "MA",  
          zip: "12345"  
}
```

```
Address { person_id: "joe",  
          street: "1 Other Street",  
          "Boston",  
          state: "MA",  
          zip: "12345"  
}
```

```
Person { _id: "joe",  
        name: "Joe Bookreader",  
        addresses: [  
            { street: "123 Fake Street",  
              city: "Faketon",  
              state: "MA",  
              zip: "12345"},  
            { street: "1 Other Street",  
              city: "Boston",  
              state: "MA",  
              zip: "12345"}  
        ]  
}
```


Model Tree Structures

- MongoDB allows various ways to use tree data structures to model large hierarchical or nested data relationships
 - Model Tree Structures with Parent References
 - Model Tree Structures with Child References
 - Model Tree Structures with an Array of Ancestors



Tree Structures – Parent References

- Presents a data model that organizes documents in a tree-like structure by storing references to *parent* nodes in *child* nodes.

```
db.categories.insert({ _id: "MongoDB", parent: "Databases" })
db.categories.insert({ _id: "dbm", parent: "Databases" })
db.categories.insert({ _id: "Databases", parent: "Programming" })
db.categories.insert({ _id: "Languages", parent: "Programming" })
db.categories.insert({ _id: "Programming", parent: "Books" })
db.categories.insert({ _id: "Books", parent: null })
```

```
db.categories.findOne({ _id: "MongoDB" }).parent
db.categories.createIndex({ parent: 1 })
db.categories.find({ parent: "Databases" })
```

Tree Structures – Child References

- Presents a data model that organizes documents in a tree-like structure by storing references to *child* nodes in *parent* nodes.

```
db.categories.insert({ _id: "MongoDB", children: [] })
db.categories.insert({ _id: "dbm", children: [] })
db.categories.insert({ _id: "Databases", children: [ "MongoDB", "dbm" ] })
db.categories.insert({ _id: "Languages", children: [] })
db.categories.insert({ _id: "Programming", children: [ "Databases", "Languages" ] })
db.categories.insert({ _id: "Books", children: [ "Programming" ] })

db.categories.findOne({ _id: "Databases" }).children
db.categories.createIndex({ children: 1 })
db.categories.find({ children: "MongoDB" })
```

Tree Structures – Array of Ancestors

- Presents a data model that organizes documents in a tree-like structure by storing references to *parent* nodes and an array that stores all ancestors.

```
db.categories.insert({ _id: "MongoDB", ancestors: ["Books", "Programming", "Databases"], parent: "Databases"})
db.categories.insert({ _id: "dbm", ancestors: ["Books", "Programming", "Databases"], parent: "Databases"})
db.categories.insert({ _id: "Databases", ancestors: ["Books", "Programming"], parent: "Programming"})
db.categories.insert({ _id: "Languages", ancestors: ["Books", "Programming"], parent: "Programming"})
db.categories.insert({ _id: "Programming", ancestors: ["Books"], parent: "Books"})
db.categories.insert({ _id: "Books", ancestors: [ ], parent: null })
```

```
db.categories.findOne( { _id: "MongoDB" }).ancestors
db.categories.createIndex({ancestors: 1})
db.categories.find({ancestors: "Programming"})
```

Aggregation

- Aggregation operations process multiple documents and return computed results.
- Aggregations are useful for:
 - Group values from multiple documents together.
 - Perform operations on the grouped data to return a single result.
 - Analyze data changes over time.
- In MongoDB the aggregation operators allows to perform analytics on your cluster without having to move your data to another platform.
 - Map-reduce

Aggregation – pipeline

- Aggregation pipeline consists of one or more stages that process documents:
 - Each stage performs an operation on the input documents
e.g. filter documents, group documents, and calculate values
 - The documents that are output from a stage are passed to the next stage.
 - An aggregation pipeline can return results for groups of documents
 - e.g. return the total, average, maximum, and minimum values
- Operations
 - projection, matching, grouping, sorting, selection

Aggregation – pipeline

Collection
↓
`db.orders.aggregate( [`
 `$match stage → { $match: { status: "A" } },`
 `$group stage → { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }`
 `]`)

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }
{ cust_id: "A123", amount: 300, status: "D" }

\$match

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }

\$group

Results	
{	<code>_id: "A123",</code>
	<code>total: 750</code>
}	
{	<code>_id: "B212",</code>
	<code>total: 200</code>
}	

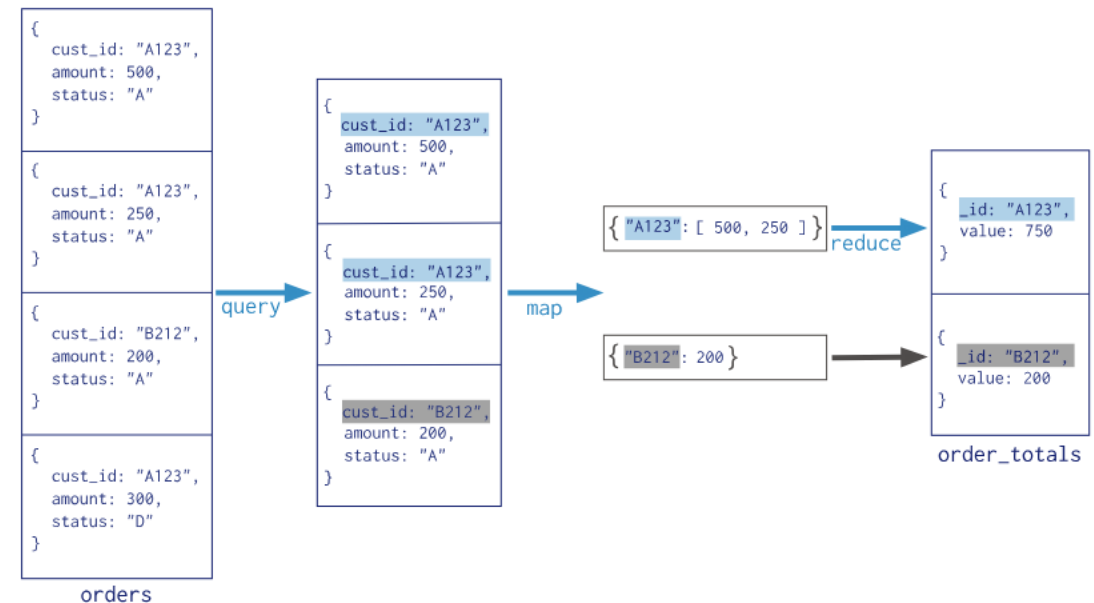
Aggregation – map-reduce

- Map-reduce is a data processing paradigm for condensing large volumes of data into useful *aggregated* results
- In mongoDB you specify
 - Map function
 - Reduce function
 - Query for data selection
 - Output format
 - Sorting criteria
 - Limits in number of obtained results

Aggregation – map-reduce

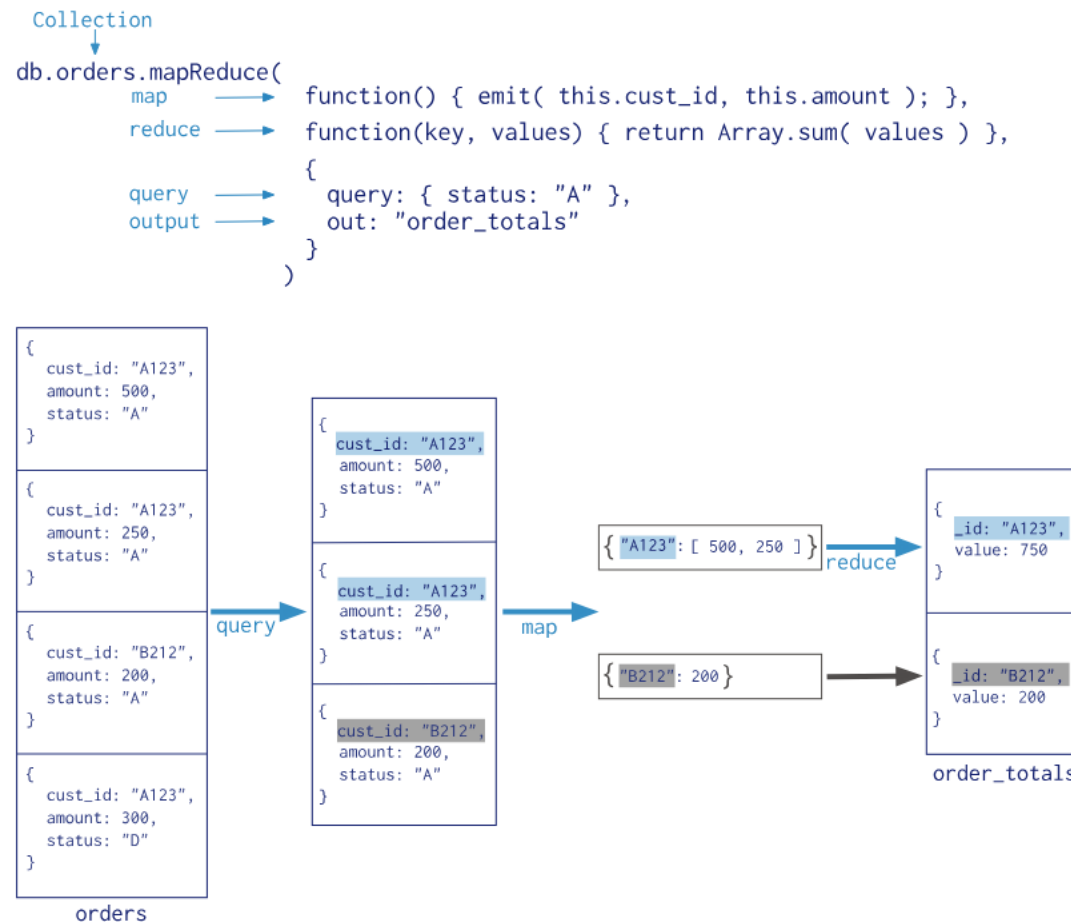
- map function is applied to each input document and returns key-value pairs.
- For keys that have multiple values, a reduce function is applied, and it collects and condenses the aggregated data.

```
Collection
↓
db.orders.mapReduce(
  map   → function() { emit( this.cust_id, this.amount ); },
  reduce → function(key, values) { return Array.sum( values ); },
  query → { query: { status: "A" },
  output → out: "order_totals"
  )
)
```



Aggregation – map-reduce

- Outputs can be stored in a collection or pass through a finalize function to further condense or process the results of the aggregation or create a new document.

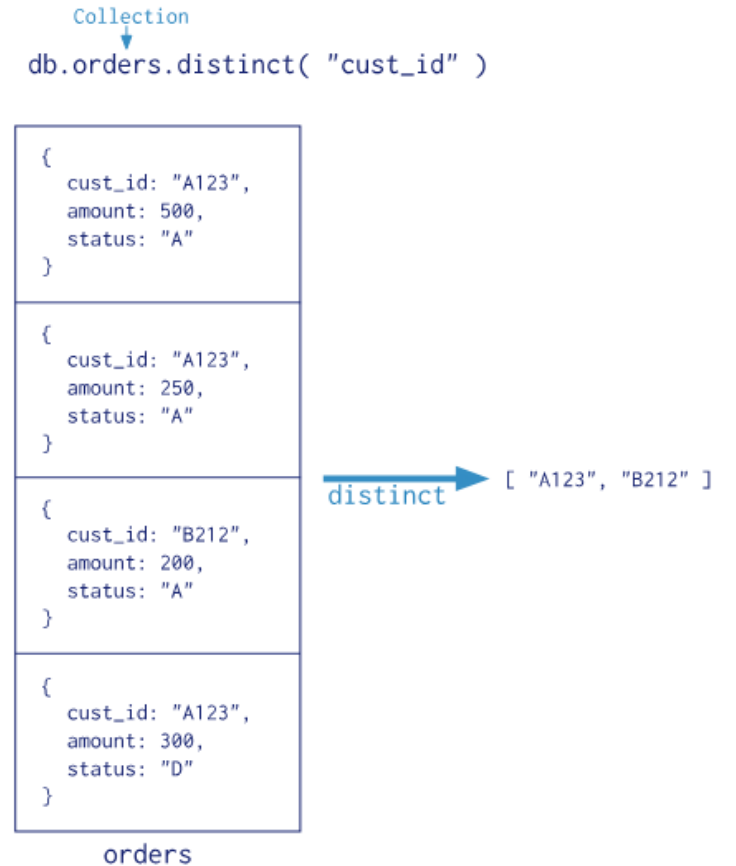


Aggregation

SQL	Aggregation
WHERE	\$match
GROUP BY	\$group
HAVING	\$match
SELECT	\$project
ORDER BY	\$sort
LIMIT	\$limit
SUM()	\$sum
COUNT()	\$sortByCount
join	\$lookup

Aggregation

- Single purpose aggregation are methods executing operations over an entire collection
 - distinct
 - count



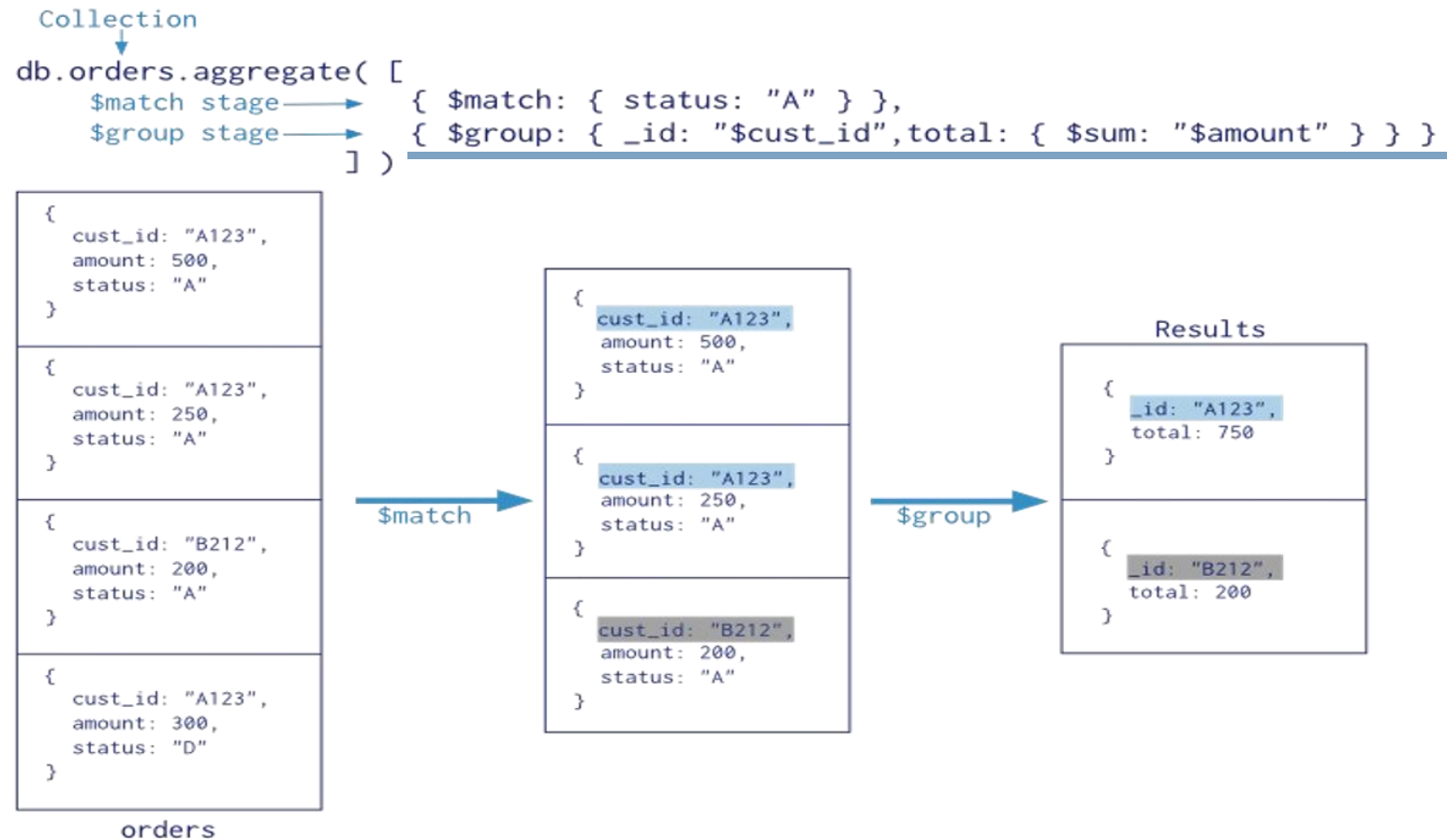
Aggregation – group

```
{ $group: { _id: <expression>, <field1>: { <accumulator1> :  
    <expression1> }, ... } }
```

- Accumulator Operators

- { \$sum: [<argument1>, <argument2> ...] }
- { \$avg: <expression> }
- { \$max: <expression> }
- { \$min: <expression> }
- { \$first: <expression> }
- { \$last: <expression> }
- ...

Aggregation – group



Aggregation – sum

sales collection:

```
{ "_id" : 1, "item" : "abc", "price" : 10, "quantity" : 2, "date" : ISODate("2014-01-01T08:00:00Z") }
{ "_id" : 2, "item" : "jkl", "price" : 20, "quantity" : 1, "date" : ISODate("2014-02-03T09:00:00Z") }
{ "_id" : 3, "item" : "xyz", "price" : 5, "quantity" : 5, "date" : ISODate("2014-02-03T09:05:00Z") }
{ "_id" : 4, "item" : "abc", "price" : 10, "quantity" : 10, "date" : ISODate("2014-02-15T08:00:00Z") }
{ "_id" : 5, "item" : "xyz", "price" : 5, "quantity" : 10, "date" : ISODate("2014-02-15T09:05:00Z") }
```

```
db.sales.aggregate( [
  { $group: { _id: { day: { $dayOfYear: "$date" }, year: { $year: "$date" } },
    totalAmount: { $sum: { $multiply: [ "$price", "$quantity" ] } },
    count: { $sum: 1 } } }
])
```

```
{ "_id" : { "day" : 46, "year" : 2014 }, "totalAmount" : 150, "count" : 2 }
{ "_id" : { "day" : 34, "year" : 2014 }, "totalAmount" : 45, "count" : 2 }
{ "_id" : { "day" : 1, "year" : 2014 }, "totalAmount" : 20, "count" : 1 }
```

Aggregation – add

sales collection:

```
{ "_id" : 1, "item" : "abc", "price" : 10, "fee" : 2, date: ISODate("2014-03-01T08:00:00Z") }  
{ "_id" : 2, "item" : "jkl", "price" : 20, "fee" : 1, date: ISODate("2014-03-01T09:00:00Z") }  
{ "_id" : 3, "item" : "xyz", "price" : 5, "fee" : 0, date: ISODate("2014-03-15T09:00:00Z") }
```

```
db.sales.aggregate([ { $project: { item: 1, total: { $add: ["$price", "$fee"] } } } ])
```

```
{ "_id" : 1, "item" : "abc", "total" : 12 }  
{ "_id" : 2, "item" : "jkl", "total" : 21 }  
{ "_id" : 3, "item" : "xyz", "total" : 5 }
```


Aggregation

```
{ cust_id: "abc123",  
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),  
  status: 'A',  
  price: 50,  
  items: [ { sku: "xxx", qty: 25, price: 1 },  
            { sku: "yyy", qty: 25, price: 1 } ]  
}
```

SQL

```
SELECT COUNT(*) AS count  
FROM orders
```

MongoDB

```
db.orders.aggregate( [ {  
  $group: { _id: null,  
    count: { $sum: 1 } }  
}])
```

Aggregation

```
{ cust_id: "abc123",  
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),  
  status: 'A',  
  price: 50,  
  items: [ { sku: "xxx", qty: 25, price: 1 },  
            { sku: "yyy", qty: 25, price: 1 } ]  
}
```

SQL

```
SELECT SUM(price) AS total  
FROM orders
```

MongoDB

```
db.orders.aggregate( [ {  
  $group: { _id: null,  
    total: { $sum: "$price" } }  
} ] )
```

Aggregation

```
{ cust_id: "abc123",  
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),  
  status: 'A',  
  price: 50,  
  items: [ { sku: "xxx", qty: 25, price: 1 },  
            { sku: "yyy", qty: 25, price: 1 } ]  
}
```

SQL

```
SELECT cust_id, SUM(price) AS total  
FROM orders  
GROUP BY cust_id
```

MongoDB

```
db.orders.aggregate( [ {  
  $group: {  
    _id: "$cust_id",  
    total: { $sum: "$price" } }  
} ] )
```

Aggregation

```
{ cust_id: "abc123",  
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),  
  status: 'A',  
  price: 50,  
  items: [ { sku: "xxx", qty: 25, price: 1 },  
            { sku: "yyy", qty: 25, price: 1 } ]  
}
```

SQL

```
SELECT cust_id, SUM(price) AS total  
FROM orders  
GROUP BY cust_id  
ORDER BY total
```

MongoDB

```
db.orders.aggregate( [ {  
  $group: {  
    _id: "$cust_id",  
    total: { $sum: "$price" }  
  } }, { $sort: { total: 1 } } ] )
```

Aggregation

```
{ cust_id: "abc123",  
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),  
  status: 'A',  
  price: 50,  
  items: [ { sku: "xxx", qty: 25, price: 1 },  
            { sku: "yyy", qty: 25, price: 1 } ]  
}
```

SQL

```
SELECT cust_id, count(*)  
FROM orders  
GROUP BY cust_id  
HAVING count(*) > 1
```

MongoDB

```
db.orders.aggregate( [ {  
  $group: {  
    _id: "$cust_id",  
    count: { $sum: 1 }  
  } },  
  { $match: { count: { $gt: 1 } } } ] )
```

Aggregation

```
{ cust_id: "abc123",  
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),  
  status: 'A',  
  price: 50,  
  items: [ { sku: "xxx", qty: 25, price: 1 },  
            { sku: "yyy", qty: 25, price: 1 } ]  
}
```

SQL

```
SELECT cust_id, ord_date,  
SUM(price) AS total  
FROM orders  
GROUP BY cust_id, ord_date
```

MongoDB

```
db.orders.aggregate( [ {  
  $group: { _id: {  
    cust_id: "$cust_id",  
    ord_date: { $dateToString: {  
      format: "%Y-%m-%d",  
      date: "$ord_date" } }  
    }, total: { $sum: "$price" } } } ] )
```